Fundamentals of AI — Problem Set 1 Report

Karthik Mattu

September 21, 2025

1 Problem 1: Decision trees for Boolean functions

Q1. Draw decision trees for given Boolean functions

1. $A \wedge \overline{B} \wedge C$.

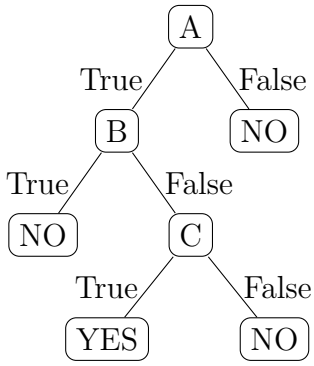

Figure 1: Decision tree for $A \wedge \overline{B} \wedge C$.

2. $(X \wedge \overline{Y} \vee \overline{X} \wedge Y)$.

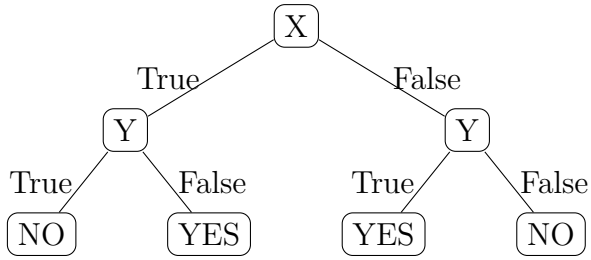

Figure 2: Decision tree for $(X \wedge \overline{Y} \vee \overline{X} \wedge Y)$.

3. $(X \wedge Y \wedge Z \vee X \wedge \overline{Y} \wedge W \vee \overline{X} \wedge Y)$.

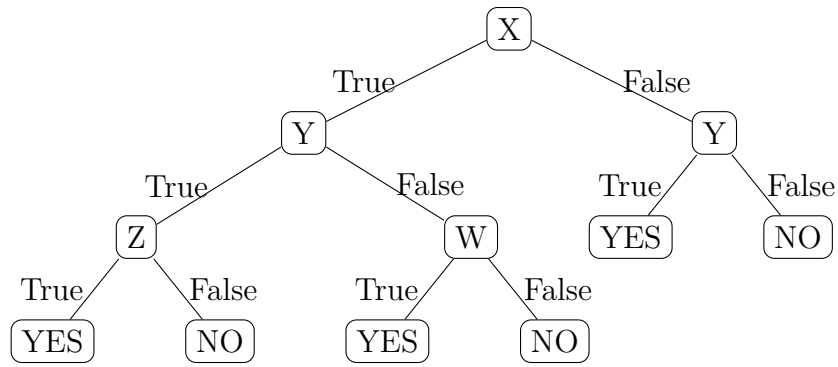

Figure 3: Decision tree for $(X \wedge Y \wedge Z \vee X \wedge \overline{Y} \wedge W \vee \overline{X} \wedge Y)$.

Q2. Root node selection using Information Gain and Gini

Provide step-by-step derivations for the provided dataset (e.g., Football). For instance, entropy of a binary label Y with $p = \Pr(Y{=}1)$:

$$H(Y) \;=\; -\,p\log_2 p \;-\; (1-p)\log_2(1-p).$$

Information gain for splitting on attribute A:

$$\mathrm{Gain}(A) \;=\; H(Y) \;-\; \sum_{v\in\mathcal{V}(A)} \frac{|Y_{A=v}|}{|Y|}\, H(Y \mid A{=}v)\,.$$

Gini impurity:

$$\mathrm{Gini}(Y) \;=\; 1 - \sum_{c} \Pr(Y{=}c)^2.$$

Calculating Information Gain

$$Entropy\ of\ whole\ set = H(Y) = H(\frac{8}{14}) = -(0.57\log_2(0.57) + 0.43\log_2(0.43))$$

$$H(Y) = H(\frac{8}{14}) = -[(0.57)(-0.81) + (0.43)(-1.22)] = 0.986$$

Now calculating Information Gain(A) for various attributes.

$$IG(League) = 0.986 - [\frac{5}{14}\ H(\frac{3}{5}) + \frac{4}{14}\ H(\frac{1}{4}) + \frac{5}{14}\ H(\frac{4}{5})]$$

$$IG(League) = 0.986 - [(0.357)(0.971) + (0.285)(0.811) + (0.357)(0.722)] = 0.150$$

$$IG(Position) = 0.986 - [\frac{6}{14}\ H(\frac{5}{6}) + \frac{4}{14}\ H(\frac{1}{4}) + \frac{4}{14}\ H(\frac{2}{4})]$$

$$IG(Position) = 0.986 - [(0.429)(0.650) + (0.286)(0.811) + (0.286)(1.00)] = 0.189$$

$$IG(Preferred\ foot) = 0.986 - [\frac{7}{14}\ H(\frac{2}{7}) + \frac{7}{14}\ H(\frac{6}{7})]$$

$$IG(Preferred\ foot) = 0.986 - [(0.50)(0.863) + (0.50)(0.591)] = 0.258$$
$$IG(Capped) = 0.986 - [\frac{7}{14}\ H(\frac{4}{7}) + \frac{7}{14}\ H(\frac{4}{7})]$$
$$IG(Capped) = 0.986 - [(0.50)(0.985) + (0.50)(0.985)] = 0.001$$

According to Information gain criteria, Root Node must be the one with maximum Information Gain, thus **Preferred foot** is the chosen root.

Now calculating Gini Impurity

$$Gini\ Index\ for\ Boolean\ classification\ (G) = 1 - (p^2 + (1-p)^2) = 2p(1-p)$$
$$Weighted\ Gini\ Index(League) = \frac{5}{14}\ G(\frac{3}{5}) + \frac{4}{14}\ G(\frac{1}{4}) + \frac{5}{14}\ G(\frac{4}{5})$$
$$Weighted\ Gini\ Index(League) = (0.357)(0.480) + (0.285)(0.375) + (0.357)(0.320) = 0.392$$

$$Weighted\ Gini\ Index(Position) = \frac{6}{14}\ G(\frac{5}{6}) + \frac{4}{14}\ G(\frac{1}{4}) + \frac{4}{14}\ G(\frac{2}{4})$$
$$Weighted\ Gini\ Index(Position) = (0.428)(0.277) + (0.285)(0.375) + (0.285)(0.500) = 0.369$$
$$Weighted\ Gini\ Index(Preferred\ foot) = \frac{7}{14}\ G(\frac{2}{7}) + \frac{7}{14}\ G(\frac{6}{7})$$
$$Weighted\ Gini\ Index(Preferred\ foot) = (0.500)(0.408) + (0.500)(0.245) = 0.326$$
$$Weighted\ Gini\ Index(Capped) = \frac{7}{14}\ G(\frac{4}{7}) + \frac{7}{14}\ G(\frac{4}{7})]$$
$$Weighted\ Gini\ Index(Capped) = (0.500)(0.489) + (0.500)(0.489) = 0.489$$

According to Gini Index criteria, Root Node must be the one with minimum Gini Index, thus **Preferred foot** is the chosen root.

2 Problem 2: Implement the decision tree algorithm

Q3: Implementation Discussion

The core structure of my decision tree implementation is based on the two classes.

- **Node Class**: internal nodes store an attribute and have a dictionary that stores the children; leaves store a class label. We use this property to distinguish between an internal node and a leaf node.

- **DecisionTree Class**: Contains all the attributes of trees such as target(ex. Diagnosis column in WDBC dataset), criterion (ig/gini) and depth(None to 15). It contains the necessary functions for building a tree and prediction for a new instance.

The tree building begins with the selection of a root node based on the selection criteria (ig/gini and depth) that provides maximum accuracy on the dev set. I have defined this step as training on 'train set' and tuning on 'dev set' The tree is then build recursively node by node following the same criteria that we followed for root node until certain limiting cases are achieved such as upon reaching leaves (nodes with a lable).Once the best criteria are found, we rebuild the tree on the train set using the best criteria. This completes our training and tuning phase.

Q4: Challenges faced

- **Recursion Implementation**: writing different stop cases was a problem and implementing nested for loops for different criteria was tricky.
- **Node class structures**: Understanding how to distinguish between an internal node and leaf node and storing children in a node was an intellectual challenge
- **Pandas Dataframe**: This was new to me and required a lot of time to learn the various methods to extract and compare different columns and rows.
- **Choosing best attribute**:- The result for Accuracy, error, precision and recall for parameters (ig,depth-2) and (gini, depth-2) are identical while tuning. There is no clear choice here so I chose (ig, depth-2).

Model	Accuracy	Error	Precision	Recall
IG-Depth-2	0.982	0.018	0.956	1.000
GINI-Depth-2	0.982	0.018	0.956	1.000

Table 1: IG vs Gini at Depth-2 performance on Dev Set

- **Server setup challenges**: It takes some time to get used to working on the server using tunneling which was new to me.

Q5: Overcoming challenges
I over came the above challenges by thoroughly understanding the pseudo code given in the book and using resources like ChatGPT and Claude to understand various nuances of Python and Dataframes. I spent a lot of time polishing my python skills especially related to handling data. The discussions I had with my peers also helped shape my approach.

3 Problem 3: Use your decision tree to develop a model for WDBC

Q6: Summary - Nuclear feature extraction for breast tumor diagnosis

The paper presents an elegant solution to the difficult problem of detecting breast cancer from Fine Needle Aspiration slides (FNAs). The approach requires sample preparation wherein a person is required to initialize 'snakes' in the vicinity of cell nucleus. These snakes then mimic the shape of the nucleus by virtue of minimizing an energy function. These digitized shapes are then processed for feature extraction. As mentioned in the paper, 10 nuclear features are extracted and additionally 3 summary stats are computed per nuclear feature and thus a total of 30 features per sample are obtained for pattern recognition.

Thereafter a MSM-tree classifier technique is used to classify breast tumor samples. Unlike a normal decision tree which splits on single feature, MSM-tree splits on linear combination of features. Thus it uses linear surfaces to separate classes. Furthermore a ten-fold cross-validation was used to estimate performance on unseen cases.

Key Observations

- Having a decision parameter that is a combination of various features leads to better accuracy for a classifier.
- A ten-fold cross validation gives a more stable estimate of how the model will perform on new, unseen patients.

Q7: Test Results

Model	Accuracy	Error	Precision	Recall
DecisionTree-IG-Depth-2 (Best)	0.930	0.070	0.854	0.976
DecisionTree-GINI	0.895	0.105	0.857	0.857
DecisionTree-IG	0.895	0.105	0.857	0.857
Majority Baseline	0.632	0.368	0.000	0.000

Table 2: WDBC test performance. Positive class: M.

Q8: Discussion

In this problem, recall is the most critical performance metric, because missing a malignant case (false negative) is far more dangerous than misclassifying a benign case as malignant (false positive).

Q9: Tuning Procedure

For the tuning procedure I made a nested for loop wherein I trained the decision tree on the training set using different hyperparameters, mainly - Splitting criteria (IG vs Gini) and Max Depth of the tree from 1 to 15 and also unlimited (none) and then evaluated these trees on the dev set, observing accuracy,error, precision, and recall. The criteria IG, Depth-2 and Gini, Depth-2 produced same performance metrics. I chose IG, Depth-2 as my best hyperparameters to retrain my decision tree on the train set. From this tuning I learned

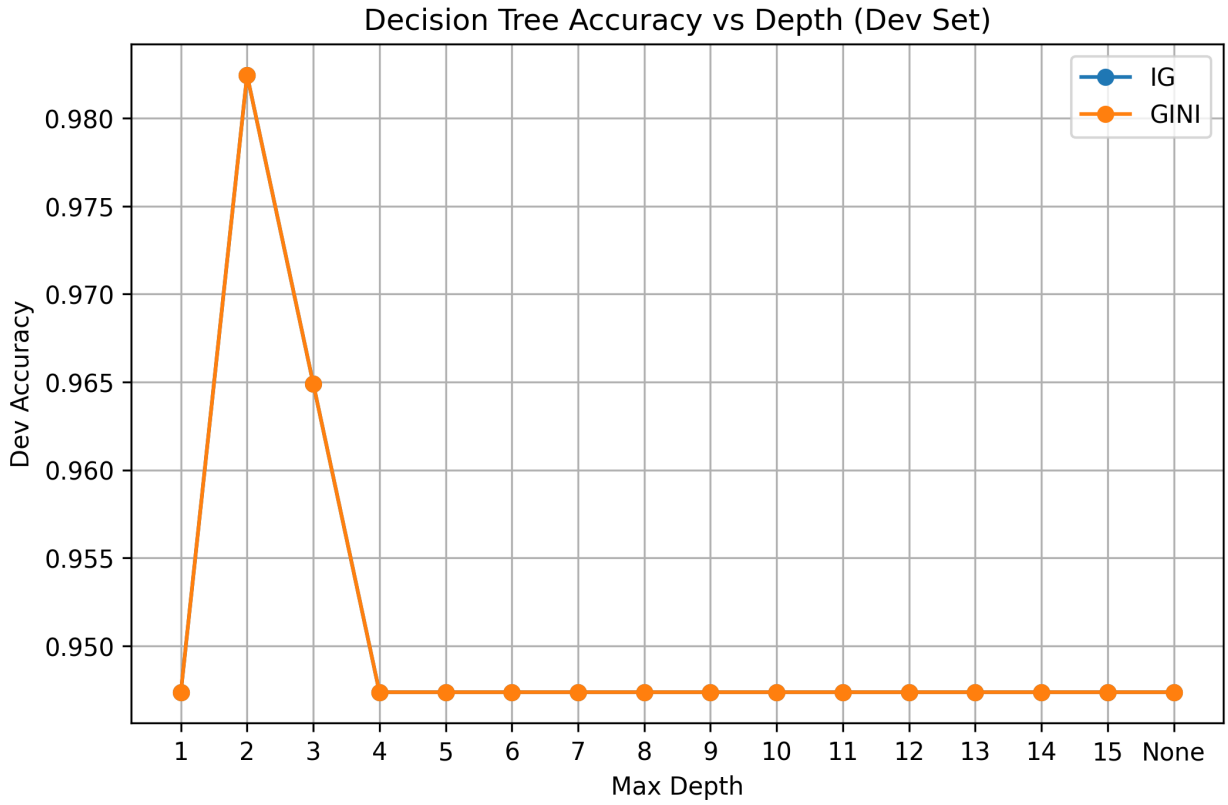

Figure 4: Accuracy vs. Depth on the Development Set for IG and Gini criteria.

about overfitting. As seen above in the plot of Accuracy vs Max Depth of the Tree - we can observe that Dev accuracy first rises when depth is increased, peaks and then drops down. This happens because for example when Depth is unlimited(None), the training procedure keeps splitting as long as it can (until a break condition is achieved) as each additional split reduces impurity. This caters well to the training set as it fits perfectly well to idiosyncracies but often hurt accuracy on unseen data.

4 Problem 4: Compare with an implementation in an ML library

Q10: Decision-tree visualization

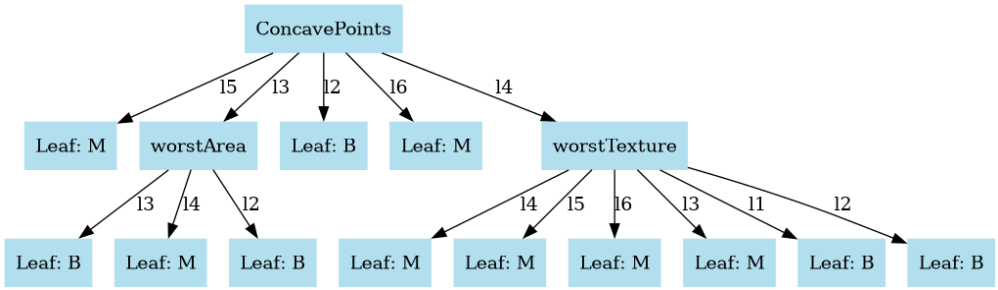

Figure 5: Custom Decision Tree visualization (IG, depth=2).

Q11: Performance comparison - Custom Decision Tree vs Skikit-learn Decision Tree

Custom Decision Tree (M only)	Accuracy	Error	Precision (M)	Recall (M)
DecisionTree-IG (Best)	0.930	0.070	0.854	0.976
DecisionTree-GINI	0.895	0.105	0.857	0.857
DecisionTree-IG	0.895	0.105	0.857	0.857
Majority Baseline	0.632	0.368	0.000	0.000

Scikit-learn Decision Tree	Precision	Recall	F1-score	Support
B (Benign)	0.96	0.97	0.97	72
M (Malignant)	0.95	0.93	0.94	42
Accuracy		0.96		114
Macro avg	0.96	0.95	0.95	114
Weighted avg	0.96	0.96	0.96	114

Table 3: Comparison of custom implementation and scikit-learn's Decision Tree on the WDBC dataset.

the best-performing model in Custom Decision Tree used Information Gain (IG) with max depth = 2, which achieved a test accuracy of 93%, precision of 0.85, and recall of 0.98 for the malignant class. The high recall indicates that this model was very effective at catching malignant cases (almost none were missed), though precision was slightly lower, meaning there were more false positives compared to true positives.

When compared with the scikit-learn decision tree (using default settings), the sklearn model achieved a slightly higher overall accuracy of 96%, with both benign and malignant classes predicted at a balanced precision–recall tradeoff (precision and recall 0.95).

It is apparent that scikit-learn decision tree is better overall with a higher accuracy and balanced precision and recall.

Q12: Comparison with scikit-learn

Similarities with the `scikit-learn` decision tree and our custom decision tree include:- Both use of information gain (entropy) and the Gini index as splitting criteria, and both tune the same hyperparameters i.e. splitting criteria (IG vs Gini) and Max Depth. Also both follow the recursive approach to build the decision tree.

Differences between the two include:- Our implementation handles only categorical splits, whereas `scikit-learn` natively supports continuous-valued features as seen by its handling of non-binned data. While `scikit-learn` reports detailed per-class metrics (precision, recall, F1-score and Support) for both benign and malignant cases, our evaluation focused more narrowly on accuracy thus different interpretations of performance: `scikit-learn` achieved more balanced metrics across both classes, whereas our implementation prioritized accuracy alone. Finally, in `scikit-learn` is automated using Graphviz (via `plot_tree`), while in our implementation we wrote a custom tree-printing method to use Graphviz.

Q13: Performance of the binned versus non-binned (normalized)

Table 4: Comparison of Binned (Custom) vs Non-binned (Sklearn) Decision Trees on Test Set

Model / Data	**Accuracy**	**Precision (M)**	**Recall (M)**
Custom (Binned, IG depth=2)	0.93	0.854	**0.976**
Sklearn (Non-binned)	**0.96**	**0.95**	0.93

The non-binned (sklearn) model achieved higher overall accuracy and precision as continuous features let sklearn choose numeric thresholds which are more expressive than fixed bins. The binned (custom) model achieved higher recall for Malignant, i.e., it missed fewer malignant cases, but at the cost of more false positives (lower precision) and lower overall accuracy.

Q14: Why the difference?

It seems that Binning/Discretization can coarsen boundaries in a way that favors catching positives (higher recall), but it loses fine-grained information that could improve classification. Furthermore Binning itself could lead to misclassification of certain cases as they could have been clubbed together with contrasting samples. This could be especially true for borderline cases. Our custom tree on binned data showed slightly lower precision but higher recall, meaning it was more likely to catch malignant cases at the expense of more

false positives. In contrast, the non-binned sklearn tree could use exact numeric thresholds, giving it better precision and overall accuracy but slightly lower recall.

Q15: Additional Hyperparameters

min_samples_split	**min_samples_leaf**	**Accuracy**	**Precision**	**Recall**
2	1	0.921	0.905	0.884
2	5	0.921	0.925	0.860
2	10	0.930	0.927	0.884
5	1	0.930	0.927	0.884
5	5	0.921	0.925	0.860
5	10	0.930	0.927	0.884
10	1	0.930	0.927	0.884
10	5	0.921	0.925	0.860
10	10	0.930	0.927	0.884

Table 5: Performance of Decision Tree with different `min_samples_split` and `min_samples_leaf` hyperparameters.

- Both hyperparameters mainly acted as regularizers to prevent overly complex trees.
- The performance differences were small, suggesting the dataset is relatively clean and separable, so depth mattered more than these hyperparameters.